An Analysis and Simulation of Defending Tracking Control Neural Network for Known Affine Discrete-Time Nonlinear Systems Against Adversarial Attacks

Ablasse Kingcaid-Ouedraogo

Last Updated: Mar 10, 2026

1 Abstract

This paper will investigate the simulation of an adversarial attack on the tracking control neural network for known affine discrete-time nonlinear system given by [1]. Additionally, the simulation will be ported to C++ and optimized. The previous simulation [1], of the system was designed in MATLAB. This paper proposes that C++ is the more optimal language for these simulations. C++ gives greater control of the hardware resources involved in the computation to the programmer, allowing for more granular optimizations to be performed.

2 Introduction

System Model

The tracking control neural network, as defined in [1], is designed to control a known affine discrete-time nonlinear system. The system is described by the following equation:

Definition 1. *Affine-In-The-Inputs Discrete-Time Nonlinear System[1]*

$$\underbrace{x(k+1)}_{\substack{\text{next}\\ \text{system-state}}} = \underbrace{x(k)}_{\substack{\text{current}\\ \text{system-state}}} + \underbrace{\tau}_{\substack{\text{discrete-time}\\ \text{step}}} \underbrace{\Big[\underbrace{f(x(k))}_{\substack{\text{state}\\ \text{dynamics}}} + \underbrace{g(x(k))}_{\substack{\text{coupling}\\ \text{dynamics}}} \underbrace{u(k)}_{\substack{\text{control}\\ \text{input}}}\Big]}_{\text{Rate of Change}} \tag{1}$$

where:

Affine-In-The-Inputs: *The system dynamics are described by the sum of the linear function of the input and the nonlinear function of the state. This property simplifies the analysis and control design.*

Discrete-Time: *The system state is updated at specific, uniform intervals of time.*

$\boldsymbol{x}$**:** *System state.*

$\boldsymbol{f}$**:** *Nonlinear function that defines the system's internal state dynamics.*

$\boldsymbol{g}$**:** *Nonlinear function that defines the effect of the control input on the state dynamics.*

$\boldsymbol{u}$**:** *The control input applied to the system.*

$\boldsymbol{\tau}$**:** *Discrete-time step.*

Control Function

The MATLAB simulation from [1] is designed to simulate the tracking control neural network. For that purpose, it is necessary to define the control function, u(k), that will be used to control the system Equation 1.

Definition 2. *Control Function* [1]

$$u(k) = \begin{cases} g(x(k))^{-1}\big(-f(x(k)) + \Delta r(k) - \beta sgn(x(k) - r(k))\big) & \text{if } g(x(k)) \neq 0 \\ 0 & \text{if } g(x(k)) = 0 \end{cases} \tag{2}$$

such that, given a reference trajectory $r(k)$, the system Equation 1 seeks to minimize the closed-loop error dynamics:

Definition 3. *Closed-Loop Tracking Control Error Dynamics* [1]

$$e(k+1) = e(k) - \beta sgn(e(k))$$

where:

$$sgn(e(k)) = \begin{cases} 1 & \text{if } e(k) > 0 \\ 0 & \text{if } e(k) = 0 \\ -1 & \text{if } e(k) < 0 \end{cases} \tag{3}$$

and

$$e(k) = x(k) - r(k)$$

Closed-Loop Tracking Control

Substituting $u(k)$ from Equation 2 into the system model Equation 1 gives the deployed closed-loop state equation:

Definition 4. *Closed-Loop Tracking Control State Equation* [1]

$$\begin{aligned} & x(k+1) = x(k) - \beta sgn(x(k) - r(k)) + \Delta r(k) \\ & \textit{where:} \\ & \Delta r(k) = r(k+1) - r(k) \end{aligned} \tag{4}$$

Functional Block-Diagram of the Controlled Nonlinear System

A functional block-diagram of the controlled nonlinear system described by Equation 1 and Equation 2 is presented in Figure 1 The diagram includes the blocks of discrete-time summation $+$, sign activation function S, amplification β, nonlinear functions $f(x)$ and $g(x)$, NN solver of system of linear algebraic equations LAESS, reference r, state variable $x(k)$ with initial value $x(1)$, and control input $u(k)$. As one can see in Fig. 1, the NN consists of $3n$ adders, n controlled switches, n amplifiers, n digital differentiators, n blocks of nonlinear functions $f(x)$, $n \times m$ blocks of nonlinear functions $g(x)$, and n NN solvers of a systems of linear algebraic equations.[1]

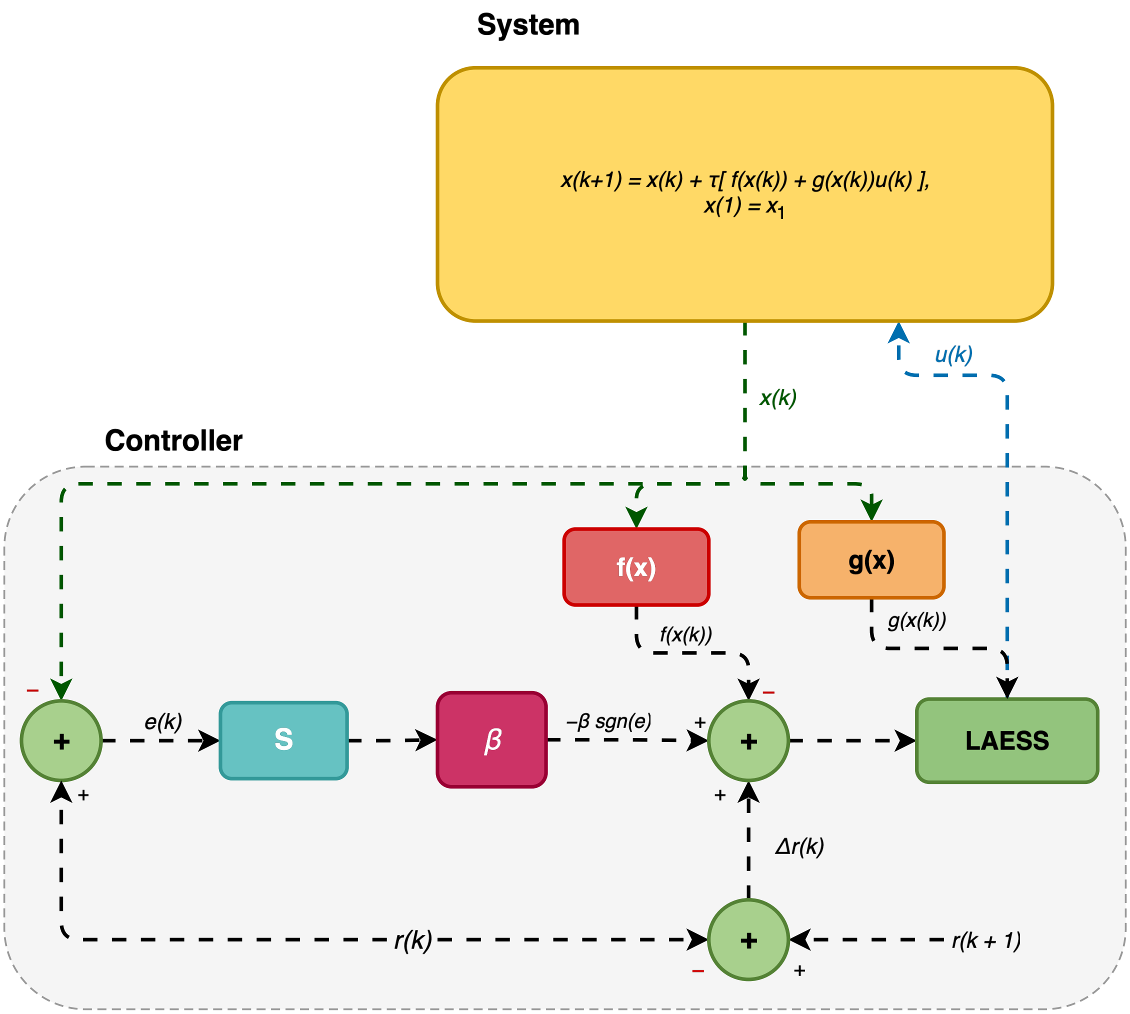


Figure 1: Functional block-diagram of the controlled nonlinear system.

3 Adversarial Attacks

For the purpose of simulating adversarial attacks against the tracking control neural network, the current state of the art method is to use a projected gradient descent (PGD)[2] attack, which is an iterative method that seeks to find the optimal perturbation that maximizes the loss function of the neural network. The simulation and defense of a neural network using the PGD method will be left for future analysis. This paper will examine the an alternative method of attack simulation, using additive gaussian noise perturbation[3]. The mathematical representation of the adversarially perturbed system is defined, considering Equation 1, by:

Definition 5. *Adversarially Perturbed Tracking State Equation*

$$\begin{aligned} & x(k+1) = x(k) + \tau\left[f(x(k)) + g(x(k))u(k)\right] + I(k) \\ & \textit{where:} \\ & I(k) \sim \mathcal{N}(0, \sigma^2 \mathcal{I}d_n) \end{aligned} \tag{5}$$

Substituting the control function from Equation 2 into the perturbed system model gives the adversarially perturbed closed-loop state equation:

Definition 6. *Perturbed Closed-Loop Tracking Control State Equation*

$$\begin{aligned} & x(k+1) = x(k) - \beta sgn(x(k) - r(k)) + \Delta r(k) + I(k) \\ & \textit{where:} \\ & I(k) \sim \mathcal{N}(0, \sigma^2 \mathcal{I}d_n) \end{aligned} \tag{6}$$

4 Theorems

Existence and Uniqueness of Steady States

Stability and Convergence of Steady States in the Presence of Adversarial Attacks

5 Figures

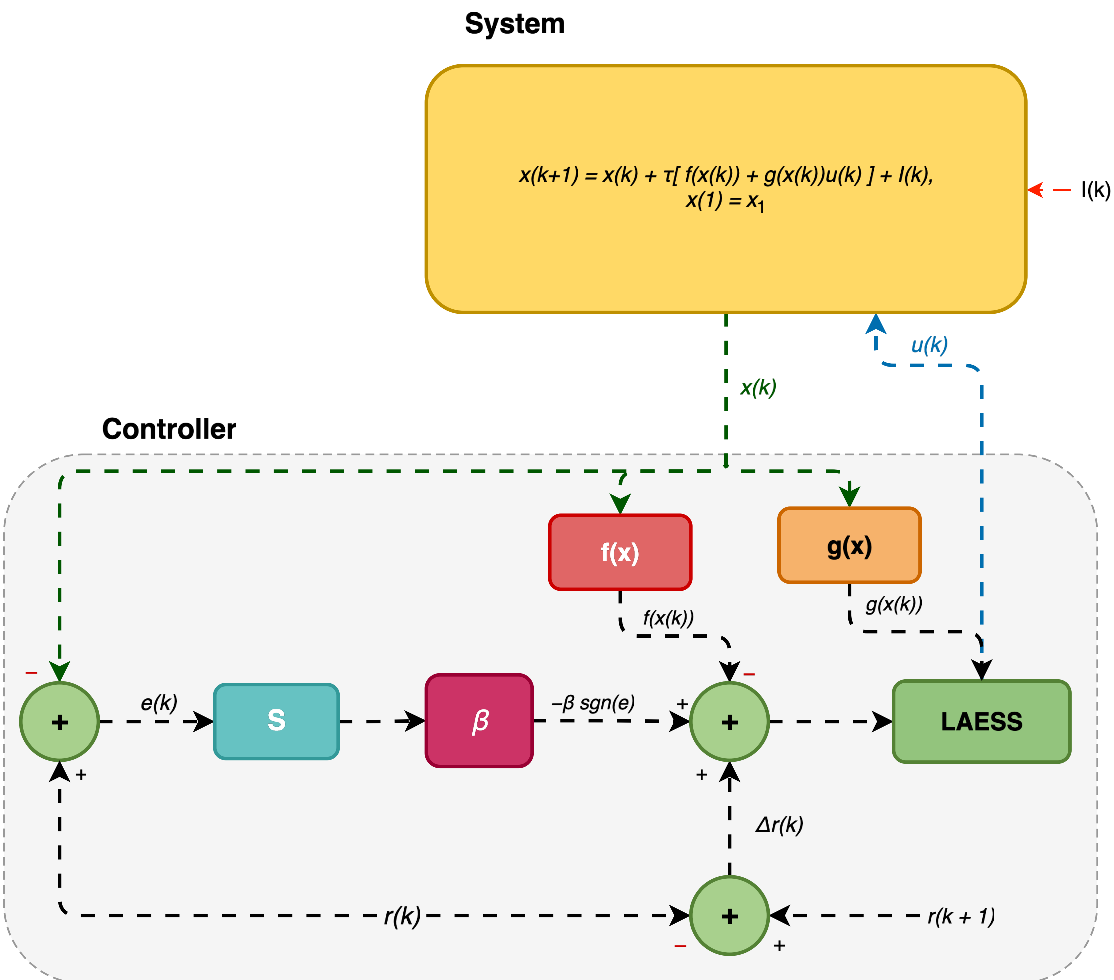


Figure 2: Functional block-diagram of the controlled nonlinear system under adversarial attack.

Figure 3: Functional block-diagram of the controlled nonlinear system with defensive measures.

References

[1] P. Tymoshchuk, "A tracking control neural network for the known affine in the inputs discrete-time nonlinear systems," tech. rep., University of North Texas, 2024.

[2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2019.

[3] H. Hajari, M. Cesaire, L. Schott, S. Lamprier, and P. Gallinari, "Neural adversarial attacks with random noises," *International Journal on*, vol. 32, p. 22, August 2023.

A Code Migration: MATLAB to C++

A.1 Motivation

The original tracking control simulation was implemented as a collection of MATLAB scripts (`Fig2a.m`–`Fig2c.m`), following the reference implementation in [1]. While MATLAB provides rapid prototyping through built-in matrix operations and plotting, several factors motivated a migration to C++:

1. **Execution speed.** The simulation loop iterates 45,000 time steps with per-step nonlinear dynamics evaluation. C++ compiled code with direct memory access eliminates the interpreter overhead inherent to MATLAB's execution model.

2. **Memory control.** C++ provides explicit control over stack versus heap allocation, cache-aligned data layouts, and zero-copy data access through pointers—critical when storing $2 \times 45{,}000$ state matrices.

A.2 Architecture Decisions

The MATLAB implementation uses a procedural style: separate 1-D arrays for each state variable (`x`, `y`, `r1`, `r2`, `e1`, `e2`, `u`) with a single `for` loop driving the simulation. The C++ port uses 2d arrays for the state variables with a single for loop as well. It adopts a struct-based architecture that groups related data and separates concerns across compilation units.

A.2.1 Data Layout

Table 1: Data structure mapping between MATLAB and C++ implementations.

MATLAB Variable	C++ Equivalent	Type
`x(1:n)`, `y(1:n)`	`stateMatrix`	`Matrix<double, 2, Dynamic>`
`r1(1:n)`, `r2(1:n)`	`referenceMatrix`	`Matrix<double, 2, Dynamic>`
`e1(1:n)`, `e2(1:n)`	`x1ErrorMatrix`, `x2ErrorMatrix`	`Matrix<double, 2, Dynamic>`
`r2(i+1)-r2(i)`	`deltaReference`	`Matrix<double, 2, Dynamic>`
`alpha`, `beta`, ...	`InternalConditions`	`struct`
`n`, `tmax`, `dt`	`SimSettings`	`struct`

The key design choice is consolidating per-variable 1-D arrays into $2 \times N$ Eigen matrices. Row 0 stores the conversion state x_1, and row 1 stores the temperature state x_2. This layout preserves temporal locality: accessing both states at time step k requires reading two adjacent memory locations in the same column.

A.2.2 Build System

The C++ project uses CMake 3.15+ with `FetchContent` to manage two external dependencies:

Table 2: C++ build dependencies fetched at configure time.

Library	Source	Version	Purpose
Eigen	GitLab	master	Matrix algebra
cxxplot	US Naval Research Lab	v0.4.1	Interactive Qt-based plotting

The build system produces three executables from separate translation units: `fig2a`, `fig2b`, `fig2c` (CSTR system). Each executable contains its own `main()` function, with shared simulation logic factored into `state_space.cpp`.

A.3 Implementation — CSTR System (Fig. 2)

This section presents side-by-side comparisons of the MATLAB and C++ implementations for each major component of the CSTR simulation.

A.3.1 System Dynamics

The CSTR system has two states: conversion x_1 and temperature x_2. Since the control gain $g_1 = 0$, the conversion state evolves under open-loop natural dynamics. The temperature state uses feedback-linearizing sliding-mode control.

Conversion state x_1 — open loop. The natural dynamics follow the discretized CSTR model:

$$x_1(k+1) = x_1(k) + \tau\left[-\alpha\, x_1(k) + D_a\big(1 - x_1(k)\big)\exp\left(\frac{x_2(k)}{1 + x_2(k)/\gamma}\right)\right] \tag{7}$$

MATLAB (`Fig2a.m`, line 34):

```
f1(i) = x(i) + dt*(-alpha*x(i) + Da*(1-x(i))*exp(y(i)/(1+y(i)/gamma)));
```

C++ (`state_space.cpp`, lines 23–28):

```
stateConditions.stateMatrix(0, i) =
    *prevX1 +
    settings.tau *
        (-internalConditions.alpha * *prevX1 +
         internalConditions.Da * (1 - *prevX1) *
             exp(*prevX2 / (1 + *prevX2 / internalConditions.gamma)));
```

The C++ version accesses parameters through the `InternalConditions` struct rather than workspace variables, and uses cached pointer variables (`prevX1`, `prevX2`) to avoid repeated matrix indexing within the inner loop.

Temperature state x_2 — closed loop. The feedback-linearizing control law cancels the nonlinear dynamics $f_2(x)$, yielding the simplified closed-loop update:

$$x_2(k+1) = x_2(k) + \tau[-\beta\ \mathrm{sgn}(e_2(k)) + \Delta r_2(k)] \tag{8}$$

where $e_2(k) = x_2(k) - r_2(k)$ is the tracking error and $\Delta r_2(k) = r_2(k+1) - r_2(k)$ is the reference change.
MATLAB (`Fig2a.m`, line 67):

```
y(i+1) = y(i) + dt*(-betta*S(i) + r2(i+1) - r2(i));
```

C++ (`state_space.cpp`, lines 31–34):

```
deltaR2 = &stateConditions.deltaReference(1, i - 1);
stateConditions.stateMatrix(1, i) =
    *prevX2 +
    settings.tau * (-internalConditions.beta * sgn(e2) + *deltaR2);
```

A notable difference is that the C++ implementation pre-computes the Δr_2 matrix in a single Eigen operation before entering the simulation loop, whereas MATLAB computes `r2(i+1)-r2(i)` inline at each step.

A.3.2 Reference Trajectory

The piecewise-constant reference switches between two operating points at $t = 15\,\mathrm{s}$ and $t = 30\,\mathrm{s}$.
MATLAB uses time-based conditionals:

```matlab
if 0<=t(i+1) & t(i+1)<=15
    r1(i+1) = 0.4472;
elseif 15<t(i+1) & t(i+1)<30
    r1(i+1) = 0.7646;
elseif 30<=t(i+1) & t(i+1)<tmax
    r1(i+1) = 0.4472;
end
```

C++ uses index-based ternary expressions:

```cpp
stateConditions.referenceMatrix(0, i) =
    i > 15000 && i < 30000 ? 0.7646 : 0.4472;
```

The index-based approach is valid because $t = i \cdot \tau$ and $\tau = 45/45000 = 0.001\,\mathrm{s}$, so $t = 15$ corresponds to $i = 15{,}000$. Additionally, the C++ version pre-computes the reference difference matrix ΔR via Eigen column operations:

```cpp
stateConditions.deltaReference =
    stateConditions.referenceMatrix.rightCols(settings.timeSteps - 1) -
    stateConditions.referenceMatrix.leftCols(settings.timeSteps - 1);
```

A.3.3 Signum Function

The signum function $\mathrm{sgn}(\cdot)$ is central to the sliding-mode controller.
MATLAB implements it with explicit branching:

```matlab
if (y(i)-r2(i)) > 0
    S(i) = 1;
elseif (y(i)-r2(i)) < 0
    S(i) = -1;
else
    S(i) = 0;
end
```

C++ uses a branchless template:

```cpp
template <typename T>
int sgn(const T &val) {
    return (T(0) < val) - (val < T(0));
}
```

The C++ implementation avoids conditional branching by exploiting boolean arithmetic: the expression evaluates to $+1$, -1, or 0 without any `if`/`else` statements, which can improve pipeline utilization on modern processors.

A.3.4 Visualization

MATLAB's native `subplot`/`plot` functions were replaced with the `cxxplot` library from the US Naval Research Laboratory. The library provides interactive Qt-based plot windows with a named-parameter API:

```cpp
auto x1Plot = cxxplot::plot(tX1, x1Ds,
    window_title_ = "Conversion",
    show_legend_  = true,
    name_         = "x1",
    xlabel_       = "Time (s)",
    ylabel_       = "x1, r1");
x1Plot.add_graph(tX1Ref, x1RefDs, name_ = "r1");
```

To handle the 45,000 data points efficiently, a min/max envelope downsampling utility reduces the data to approximately 2,000 points while preserving visual fidelity of peaks and transients.

Table 3 maps the MATLAB plotting calls to their C++ equivalents.

Table 3: Visualization API mapping.

MATLAB	C++ (cxxplot)
`plot(t, x, 'k-')`	`cxxplot::plot(t, x)`
`xlabel('Time (s)')`	`xlabel_ = "Time (s)"`
`ylabel('x_1, r_1')`	`ylabel_ = "x1, r1"`
`legend('x_1', 'r_1')`	`show_legend_ = true`, `name_ = ...`
`subplot(2,1,k)`	separate `cxxplot::plot()` call per window

A.4 Performance Comparison

The C++ simulation includes instrumentation via `std::chrono::high_resolution_clock`. The MATLAB simulation uses `tic`/`toc`. Both implementations were executed 20 times and averaged. Figure 4 compares execution times across implementations.

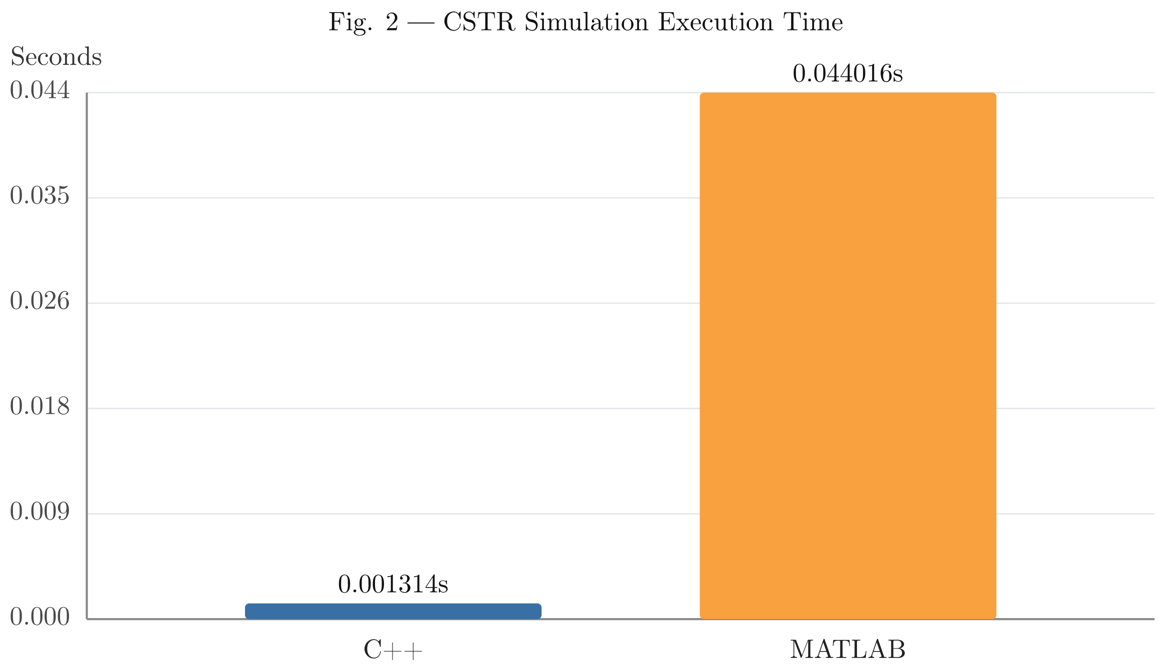
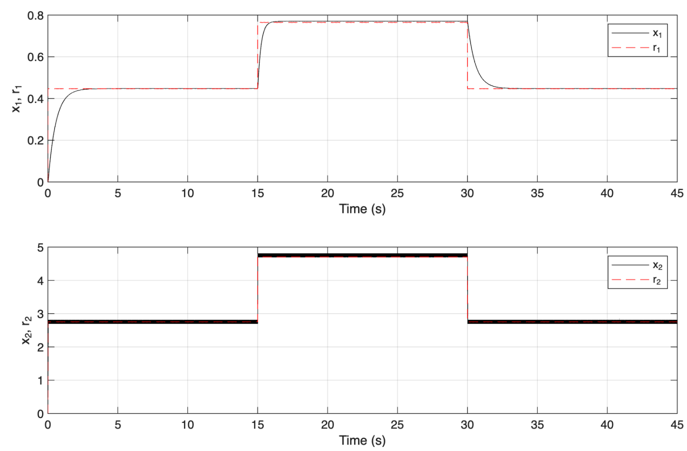


Figure 4: Execution time comparison for the CSTR tracking control simulation (n = 45,000 steps, τ = 0.001 s). Average over 20 runs.

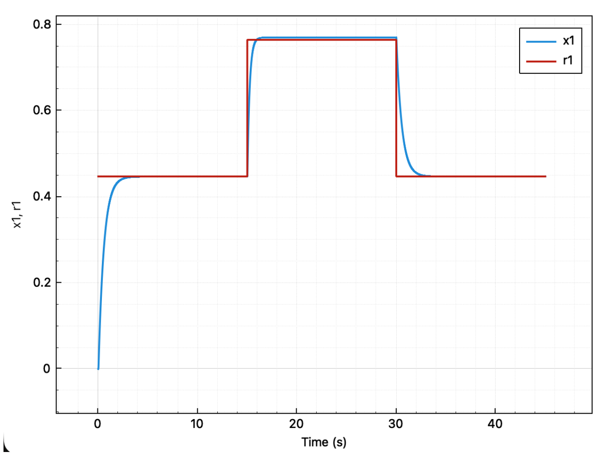
Note that the MATLAB averages include a first-run JIT compilation overhead, which is representative of interactive usage. Excluding the warmup run, the steady-state MATLAB time is approximately 0.005 s, reducing the speedup factor to approximately 4×. The JIT overhead is a real cost in the MATLAB execution model and is therefore included in the reported averages.

A.4.1 Output Comparison — CSTR System

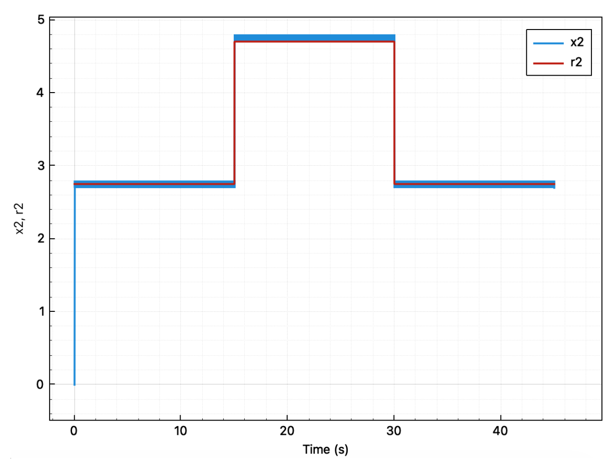
Figure 5 presents side-by-side output from both implementations to verify numerical equivalence.



(a) MATLAB (`Fig2a.m`).



(b) C++ (`fig2a`, conversion x_1).



(c) C++ (`fig2a`, temperature x_2).

Figure 5: Comparison of CSTR state trajectories from the MATLAB reference implementation and the C++ port. Both simulations use identical parameters ($\alpha = 1$, $\beta = 100$, $\lambda = 0.3$, $\gamma = 20$, $D_a = 0.072$) and initial conditions $x(0) = \mathbf{0}$.

A.5 Output Validation

Verifying numerical equivalence between two independent implementations requires more than visual comparison of plots. The migration includes a CSV-based validation pipeline that compares the C++ simulation output against MATLAB reference data at every time step.

A.5.1 Validation Pipeline

Both implementations export identical 7-column CSV files containing $\big(t,\ x_1,\ x_2,\ r_1,\ r_2,\ e_1,\ e_2\big)$ at each of the $n = 45{,}000$ time steps. The MATLAB script writes its reference data via `writematrix` at the end of the simulation loop; the C++ executable writes its output through `std::ofstream` with 15-digit precision.

A standalone validation executable (`validate_output`) loads both CSV files, extracts corresponding columns into Eigen vectors, and computes the element-wise maximum absolute error for each variable:

$$\varepsilon_{\max}^{(v)} = \max_{k=0}^{n-1} \big| v_{\mathrm{C++}}(k) - v_{\mathrm{MATLAB}}(k) \big|, \quad v \in \{x_1, x_2, e_1, e_2\} \tag{9}$$

Each variable passes if $\varepsilon_{\max}^{(v)} < \epsilon$ for a configurable tolerance ϵ. The validator reports per-variable pass/fail status and returns a nonzero exit code on any failure, enabling integration into automated build pipelines.

A.5.2 Validation Results

After correcting an off-by-one initialization error in the reference trajectory (see §A.3.2), the validation confirms machine-precision agreement between the two implementations:

Table 4: Maximum absolute error between C++ and MATLAB outputs for the CSTR tracking control simulation ($n = 45{,}000$, $\tau = 0.001\,\mathrm{s}$).

Variable	$\varepsilon_{\max}$	Status
x_1 (conversion)	$\sim 10^{-15}$	PASS
x_2 (temperature)	$\sim 10^{-15}$	PASS
e_1 (tracking error)	$\sim 10^{-15}$	PASS
e_2 (tracking error)	$\sim 10^{-15}$	PASS

The errors at $\mathcal{O}(10^{-15})$ are consistent with IEEE 754 double-precision machine epsilon ($\approx 2.22 \times 10^{-16}$), confirming that the two implementations compute identical floating-point results up to rounding in the final digits. The default tolerance of $\epsilon = 10^{-6}$ provides a margin of nine orders of magnitude above the observed error floor.

A.6 Remaining Work

The following component has not yet been migrated from MATLAB to C++:

1. **Adversarial attack simulations.** PGD attacks on the temperature measurement channel and neural network approximation-based attacks (planned implementations using IBM ART).